Aula 24 - Redes Industriais e Robótica

Descrição: Fundamentos da configuração de redes de computadores e conceitos de parametrização robótica em IoT.

Objetivos de Aprendizagem

- Compreender a arquitetura Cliente-Servidor, a pilha TCP/IP (DHCP, DNS, NAT) e a evolução para IPv6/6LoWPAN.
- Diferenciar os protocolos de aplicação HTTP, CoAP e MQTT (arquitetura *Broker*) e analisar a convergência OT/IT por meio de redes determinísticas.
- Identificar as 4 unidades de um objeto inteligente, aplicando conceitos de computação física, varredura em sistemas embarcados e prevenção de falhas elétricas (conversão de níveis lógicos) em protoboard virtual.
- Parametrizar conectividade estática (MAC, IP fixo, Máscara e Gateway) e instanciar um servidor de borda HTTP na porta 80 para processar *Request Parameters*.
- Integrar sensores ambientais virtuais (LDR/DHT11) e parametrizar um atuador robótico (Servomotor de 0° a 180°) via rede simulada.

Módulo 1: Fundamentos de Redes, Protocolos e Convergência para IoT

1.1 Arquitetura Cliente-Servidor e Serviços de Rede (DHCP, DNS e NAT)

Na base da pilha TCP/IP, a comunicação na internet e nas redes locais apoia-se na arquitetura **Cliente-Servidor** (vale destacar que mesmo protocolos orientados a mensageria na nuvem, como o MQTT, utilizam essa base na camada de transporte, onde o *Broker* atua como o servidor central e os nós IoT atuam como clientes que se conectam a ele):

- **Servidor:** Software ou dispositivo que mantém uma porta de comunicação aberta à espera de conexões. Deve estar permanentemente disponível e possuir um endereço conhecido na rede.
- **Cliente:** Dispositivo ou aplicação que inicia a comunicação de forma ativa (automática ou acionada pelo usuário) para consumir dados ou enviar comandos ao servidor.

Para que os nós conversem na infraestrutura de rede, três serviços essenciais operam conjuntamente:

1. **DHCP (*Dynamic Host Configuration Protocol*):** Automatiza a configuração de rede, enviando um endereço IP disponível, máscara de sub-rede e gateway assim que o dispositivo é ligado. *(Atenção: em servidores de borda industriais, o DHCP é substituído por IP Estático para garantir que o endereço do equipamento nunca mude)*.
2. **DNS (*Domain Name System*):** Realiza a tradução de nomes legíveis (como `broker.industria.local`) em endereços IP numéricos compreensíveis pela rede.
3. **NAT (*Network Address Translation*):** Executado pelo roteador para mapear endereços IP privados internos em IPs válidos públicos, resolvendo limitações estruturais de roteamento.

1.2 Do IPv4 ao IPv6: O Padrão 6LoWPAN e o Roteamento RPL

O crescimento exponencial de dispositivos conectados levou ao esgotamento dos endereços **IPv4** (32 bits), tornando obrigatória a adoção do **IPv6** (128 bits) para atender à imensa escala da Internet das Coisas (IoT).

Como microcontroladores e sensores possuem severas restrições de energia e largura de banda, o grupo **6LoWPAN** (*IPv6 over Low-Power Wireless Personal Area Networks*) padronizou a compressão e adaptação do IPv6 para operar sobre o padrão de rádio **IEEE 802.15.4** (redes sem fio pessoais de baixa potência).

- **O Roteamento RPL (*Routing Protocol for Low-Power and Lossy Networks*):** Gerencia o tráfego interno dessas redes de baixa potência e com perdas (LLNs).
- **Topologia DODAG (*Destination-Oriented Directed Acyclic Graph*):** O protocolo RPL organiza os nós em um Grafo Acíclico Direcionado Orientado ao Destino, calculando de forma robusta e dinâmica a melhor rota de comunicação em direção à estação base (*root/border router*) sem gerar loops de pacotes.

1.3 Protocolos da Camada de Aplicação: HTTP, CoAP e MQTT

- **HTTP (*Hypertext Transfer Protocol*):** Opera no modelo clássico de requisição e resposta. Foi desenvolvido originalmente para computadores pessoais (PCs), contando com cabeçalhos extensos em texto puro. Em IoT, é útil para servidores locais de configuração ou páginas web na borda, mas consome muitos recursos estruturais.
- **CoAP (*Constrained Application Protocol*):** Alternativa leve ao HTTP, projetada para nós com recursos computacionais severamente limitados, operando sobre UDP com baixa sobrecarga.
- **MQTT (*Message Queuing Telemetry Transport*):** Projetado especificamente para ambientes de IoT e telemetria. Baseia-se no padrão **Publicar/Assinar (*Publish/Subscribe*)** intermediado por um servidor central denominado **Broker** (hospedado localmente ou na nuvem). Os sensores publicam dados no *Broker* e os atuadores ou aplicações leem esses registros de forma desacoplada, evitando sobrecarregar o microcontrolador.

Conexão Prática (Quando usar MQTT vs. HTTP?): Enquanto o **MQTT (Publish/Subscribe)** é o padrão ouro da indústria para telemetria contínua e escalável em larga escala (muitos dispositivos enviando dados para um Broker central), o modelo HTTP Cliente-Servidor direto no microcontrolador (Porta 80) - implementado nas práticas de borda - é amplamente utilizado para parametrização direta de bancada, páginas internas de diagnóstico e controle local ponto a ponto via IP fixo, sem depender de infraestruturas externas de mensageria.

1.4 Convergência OT/IT nas Redes Industriais

No chão de fábrica da Indústria 4.0, unificamos dois domínios tradicionalmente isolados:

- **OT (*Operational Technology* - Tecnologia de Operação):** Redes de automação industrial focadas em malhas de controle (*control loops*). Exigem **redes determinísticas**, nas quais a comunicação deve ocorrer estritamente dentro de prazos rigorosamente definidos (janelas de microssegundos ou milissegundos).
- **IT (*Information Technology* - Tecnologia da Informação):** Infraestrutura corporativa, servidores globais, bancos de dados e ferramentas em nuvem.
- **A Convergência:** Viabiliza que a telemetria crítica em tempo real capturada no chão de fábrica (OT) converse de forma segura e transparente com as plataformas corporativas e dashboards analíticos globais (IT).

Módulo 2: Arquitetura de Dispositivos e Computação Física

2.1 As 4 Unidades de um Objeto Inteligente

Todo dispositivo IoT é arquitetado sobre quatro blocos fundamentais:

1. **Unidade de Processamento e Memória:** Microcontrolador (MCU) que executa os algoritmos locais, armazena variáveis e gerencia a lógica de controle.
2. **Unidade de Comunicação:** Interfaces sem fio (Wi-Fi, Bluetooth, IEEE 802.15.4) ou cabeadas (Ethernet/RS-485) que conectam o objeto à rede.
3. **Fonte de Energia:** Baterias, fonte chaveada via rede elétrica ou sistemas de colheita de energia (*energy harvesting*).
4. **Interface de Sensores e Atuadores:** Componentes que monitoram grandezas físicas (sensores) e agem mecanicamente ou eletricamente sobre o ambiente (atuadores).

2.2 Computação Física e Particularidades dos Sistemas Embarcados

A **Computação Física** integra hardware e software para construir sistemas interativos capazes de sentir e responder ao mundo analógico:

- **Entrada Analógica → Digital:** Sensores convertem fenômenos físicos (luz, calor, umidade) em variações de tensão ou corrente elétrica lidas pelo conversor Analógico-Digital (ADC) do microcontrolador.
- **Processamento → Saída Física:** O software processa os valores e aciona saídas (como sinais PWM) para controlar motores, relés e válvulas.

Particularidades do Software Embarcado:

- **Restrição de Recursos:** Memória RAM (poucos Kilobytes) e processamento reduzidos exigem código enxuto e manipulação eficiente de tipos de dados.
- **Operação Autônoma e Loop Infinito:** Como operam sem intervenção humana e em tempo real, o programa nunca deve "terminar". Utiliza-se um **laço de repetição infinito** — como `for(;;)` em C padrão ou a função `void loop()` na estrutura Arduino/ESP32 — para varrer continuamente os periféricos e a porta de rede.

Módulo 3: Prática Guiada — Parametrização de Rede e Atuação Robótica

Nesta atividade prática, você construirá um **Servidor de Borda Industrial Simulado**. Ele lerá a luminosidade ambiente com um sensor **LDR**, operará com configuração de **Rede Estática** na porta **80** e permitirá parametrizar remotamente o ângulo exato de um **Servomotor (0° a 180°)** via requisições HTTP (*Request Parameters*).

Passo 1: Preparação do Ambiente Virtual no Wokwi

1. Acesse wokwi.com e inicie um novo projeto com **ESP32**.
2. Adicione e conecte ao seu esquemático virtual:
   - `1x` **Fotoresistor (LDR) Sensor Module** (Módulo LDR de 3 ou 4 pinos).
   - `1x` **Servo** (Servomotor padrão de 180°).
   - `1x` **Half Breadboard** (Protoboard para organização dos barramentos de energia).

Passo 2: Código-Fonte Completo (Servidor de Borda + Telemetria + Robótica)

Cole o código anexo no editor principal do simulador. Analise atentamente os comentários de cada bloco.

Módulo 4: Roteiro de Validação e Desafio Prático

Para validar a aula e comprovar o domínio sobre a parametrização de rede e controle robótico, execute os **3 passos de validação** abaixo e entregue os resultados solicitados.

Etapa A: Customização Individual de Endereçamento (Obrigatório)

Para garantir que cada estação de trabalho na rede industrial possua um endereço único, personalize o seu código antes de rodar a simulação. No IP use 100 + final da matrícula (ex.: 123), e no último byte do MAC mantenha apenas os dois dígitos precedidos de 0x (ex.: 0x23)

Etapa B: Teste de Variação Ambiental e Parametrização Robótica

1. Inicie a simulação no Wokwi (botão verde *Play*).
2. Clique sobre o componente **LDR** durante a simulação e mova o controle deslizante (*slider* de Lux) para alterar a iluminação ambiente.
3. No campo de digitação do **Monitor Serial**, envie os seguintes pacotes de requisição HTTP simulando o cliente na rede:
   - Digite `GET /?angulo=45 HTTP/1.1` e pressione Enter. Observe o braço do servomotor girar exatamente para **45°**.
   - Digite `GET /?angulo=180 HTTP/1.1` e pressione Enter. Observe o servomotor atingir o fim de curso em **180°**.
   - Digite `GET /?angulo=0 HTTP/1.1` e pressione Enter para retornar à posição inicial (**0°**).

Etapa C: Desafio de Engenharia - Trava de Segurança Física-Lógica (*Control Loop*)

Em uma planta industrial real (OT), certas válvulas robóticas não podem ser abertas totalmente se o ambiente estiver no escuro (manutenção ou falha de energia no galpão).

Sua Missão: Modifique a lógica do código para criar uma **regra de intertravamento**:

- Se a luminosidade lida pelo **LDR for inferior a 20%** (`luminosidadePct < 20`), o sistema deve **bloquear qualquer comando de rede que tente abrir o servomotor acima de 90°**, limitando o eixo em no máximo `90°` e imprimindo um alerta de segurança no console: `[ALERTA OT] Baixa luminosidade! Abertura limitada a 90 graus.`

Checklist de Entrega Final

1. **Link público do projeto no Wokwi** contendo o circuito montado e o código modificado com seu IP exclusivo e a Trava de Segurança funcionando.
2. **Respostas curtas no cabeçalho do código (em formato de comentário `/* ... */`):**
   - ***Pergunta 1**:* Por que utilizamos a configuração de IP Estático neste projeto em vez de deixar o protocolo DHCP atribuir o endereço automaticamente?
   - ***Pergunta 2**:* Qual a diferença técnica entre acionar um motor DC comum e parametrizar um Servomotor para atuar em uma válvula industrial?
   - ***Pergunta 3**:* O que aconteceria em uma montagem física real se conectássemos um sensor que envia sinal lógico de 5V diretamente em uma porta GPIO de 3.3V sem conversão de nível lógico?